

Raytrace: remove repeated Python work

The verified shadow-ray change lowers target-VM median runtime by 22.63%, from 789.76 to 611.02 ms per 100 x 100 frame. The new full exact kernel adds scalar sphere intersections, camera reuse, direct nearest-hit selection and other object/call reductions. Its local Mac experiment is promising, but a full-kernel Ubuntu result is still required.

Evidence / candidate	Result	Decision
Ubuntu shadow_ray	22.63% lower runtime; pixel identity	Measured improvement above 7%.
Local Mac full	38.55% lower median, exploratory	New exact default; VM measurement pending.
Local Mac full_slots	39.81% lower median, exploratory	Optional: broader object-model restrictions.
FP64 accelerator	Complete RTL; syntax/elaboration checked	Analytical proposal; no achieved speedup.

The benchmark renders a deterministic scene with seven spheres, a halfspace, two point lights, materials, shadows and recursive reflected rays. For each pixel it creates a primary ray, finds intersections, selects the closest valid hit and computes shading/reflections. The exact upstream behavior, including its geometric and checkerboard quirks, is the reference; correcting that behavior would change the benchmark.

Vector, Point, Ray, Sphere, Halfspace and scene/material objects form a graph of ordinary Python objects. Coordinates are boxed Python floats accessed through attributes; object lists drive intersection loops. Canvas stores byte colour data with array. Dependencies are Python math/array and pyperf for the official harness. There is no NumPy or native numeric kernel in the final software.

The baseline wrapper invokes `upstream.bench_raytrace` itself, including its scene setup and render loop. Image-file writing is outside the upstream timer and is enabled only for checks. The optimized variant installs selected methods temporarily and restores them afterward. The original source and baseline wrappers were not changed during this work.

Measured pair: `raytrace_baseline_20260920-012836` and `raytrace_optimized_20260920-014248`. Comparison: `comparison_raytrace_20260920-015451`. All are under results/.

Measure the workload before changing it

The target is Ubuntu 22.04.5, Linux 5.15.0-1106-kvm, one virtual CPU exposing an Intel Xeon E5-2630 v3 at 2.40 GHz. Clean timing uses release CPython 3.10.12, CPU 0 affinity, a fixed hash seed and cyclic GC disabled. The VM governor is unavailable; host scheduling, frequency and contention are not fully controlled. These are measurements of this guest, not bare-metal Haswell limits.

Instrument	What it establishes	What it cannot establish
Clean timing	11 fresh processes per arm; upstream internal timer; every observation retained.	No proof against between-block drift.
perf stat	Separate repeated counting runs; instructions, cycles, branches and loads per equal work.	Its elapsed time is not the clean timing result.
perf record + FlameGraph	Native debug stacks, fixed 5-million-cycle sampling period; aggregate all folded stacks.	Not a release-time percentage or a count of Python calls.
cProfile	Exact call counts and Python function attribution in a separate run.	Tracing time is perturbed and is not a speedup measurement.
pyperformance / pyperf	Official suite baseline cross-check in its worker harness.	No official optimized-suite result is present.

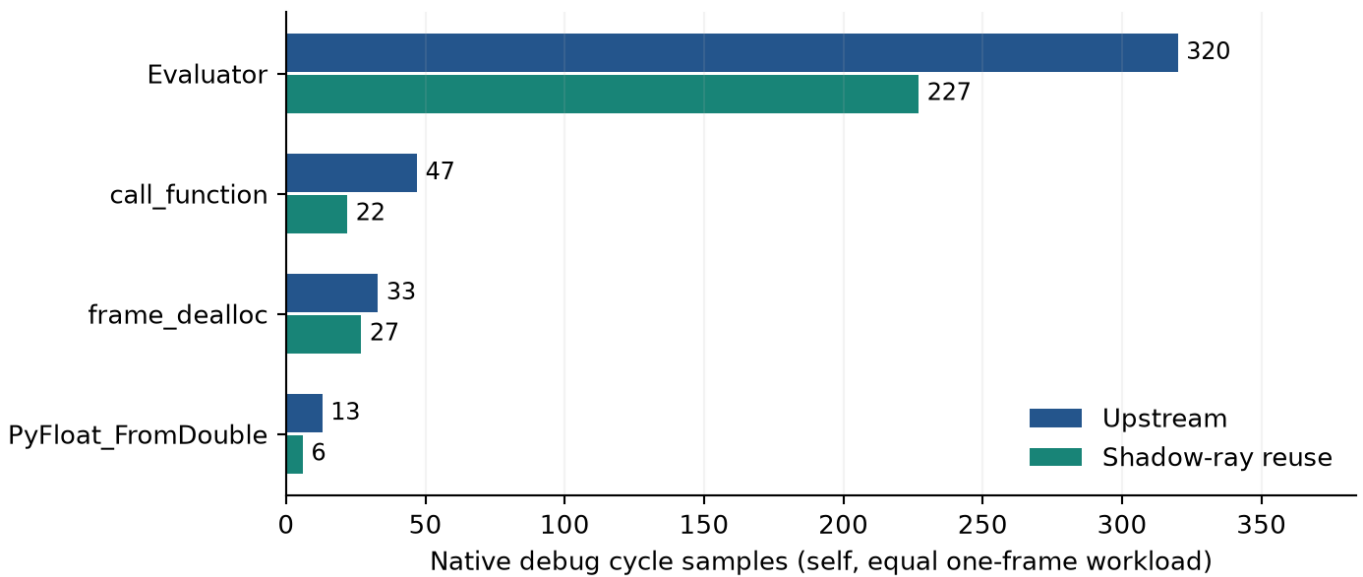
The custom wrapper calls the actual upstream bench function and uses its timer. A minimal pyperf import stub exposes `time.perf_counter`; it does not implement `pyperf.Runner`. Thus this is an upstream-workload experiment through a custom harness, alongside a separate official pyperformance baseline run. That distinction does not invalidate the experiment; the workload, timer boundaries and comparison controls must be explicit.

Runtime reduction = $100 \times (1 - \text{optimized median} / \text{baseline median})$. Speedup = $\text{baseline median} / \text{optimized median}$. We use runtime reduction for the declared 7% target. CV = $\text{sample standard deviation} / \text{mean}$. Samples from separate TXT and CSV perf invocations are not pooled.

From stacks to a testable hypothesis

A flame graph is an aggregation of sampled call stacks, not a timeline. Width is inclusive sampled weight; height is stack depth, not elapsed time. Self weight counts only samples whose leaf is that symbol. Children weight includes descendants and overlaps other ancestors. The libc start routine can therefore have a huge inclusive share while doing little direct work. Aggregate a function across every folded stack: the top-30 listing is incomplete, and repeated appearances must not be mistaken for separate causes.

The repeating evaluator/vectorcall chain reflects nested Python method execution. Tall similar stacks alone cannot count calls or price each call. cProfile supplies the stronger mechanism: one frame has 509,873 dot calls, 277,865 Point.__sub__/isPoint calls and 179,457 sphere intersection calls. In total it records 3,092,298 calls, including non-Python calls. A fixed claim of 1,600 machine instructions for every call is not supported.



Equal work: one frame per native debug profile, period 5 million cycles. These selected rows are self counts. Total samples: 1,073 baseline and 823 shadow-ray. Counts are indicative; sparse differences are not precise estimates.

The actionable question is whether identical geometry is repeatedly wrapped in new objects. In `lightIsVisible`, the same origin and light direction are used while testing successive objects. Reconstructing and normalizing that ray for every object repeats work without changing the ray. Reusing it once per visibility query predicts fewer Ray/Vector creations, normalizations, dot products and guard calls, while leaving the number of intersection queries unchanged.

CPython is a bytecode interpreter, not an AST interpreter. `_PyEval_EvalFrameDefault` includes bytecode execution, operand handling, loop control and reference work; its self share is not pure call overhead. Debug checks and different code generation change costs. High IPC and few cache misses support an instruction-work hypothesis but do not identify a precise hardware stall limit. Reject negative top-down fractions and unsupported PMU events rather than interpreting them as valid measurements or zeros.

Shadow reuse: the measured first change

`shadow_ray` builds one ray per visibility query and reuses it for each object test. This requires read-only intersection methods; custom or mutating implementations use the original path. It retains object traversal order, short-circuit behavior, comparison thresholds and upstream handling of intersections beyond the light. It does not introduce a different visibility rule.

One-frame cProfile count	Upstream	<code>shadow_ray</code>	Interpretation
Ray construction	98,172	26,000	72,172 duplicate constructions removed.
Vector construction	452,955	308,611	144,344 fewer objects; import overhead in both.
normalization / magnitude	109,887	37,715	Same removed ray-normalization work.
dot	509,873	437,701	Arithmetic helper calls decrease.
Sphere intersection	179,457	179,457	Same geometric query count.
<code>lightsVisible</code>	10,666	10,666	Same visibility queries.
All recorded calls	3,092,298	2,299,196	25.65% fewer calls; not a runtime fraction.

This combination is stronger evidence than a large interpreter percentage: a concrete source change removes a known repeated computation; exact call counts change by the predicted amount; equal-work instructions and clean runtime both fall. It supports redundant Python work as an important cost, without claiming a single universal bottleneck.

Pixel identity was checked at 24 x 24, 100 x 100 and 37 x 23. The 100 x 100 PPM SHA-256 prefix is 11f2c36b02044f11. Current verification checks every frame in the requested batch, independent sizes, and baseline restoration. A matching image alone is supplemented by raw colour, geometry and randomized tests for the new full kernel.

Historical shadow artifacts do not fully capture the optimized source hash, and manifests record a dirty working tree. The workload records and saved tests support the result, but do not reconstruct every historical local edit. Current workflows capture source hashes and explicit session paths.

The full exact kernel and the decision ledger

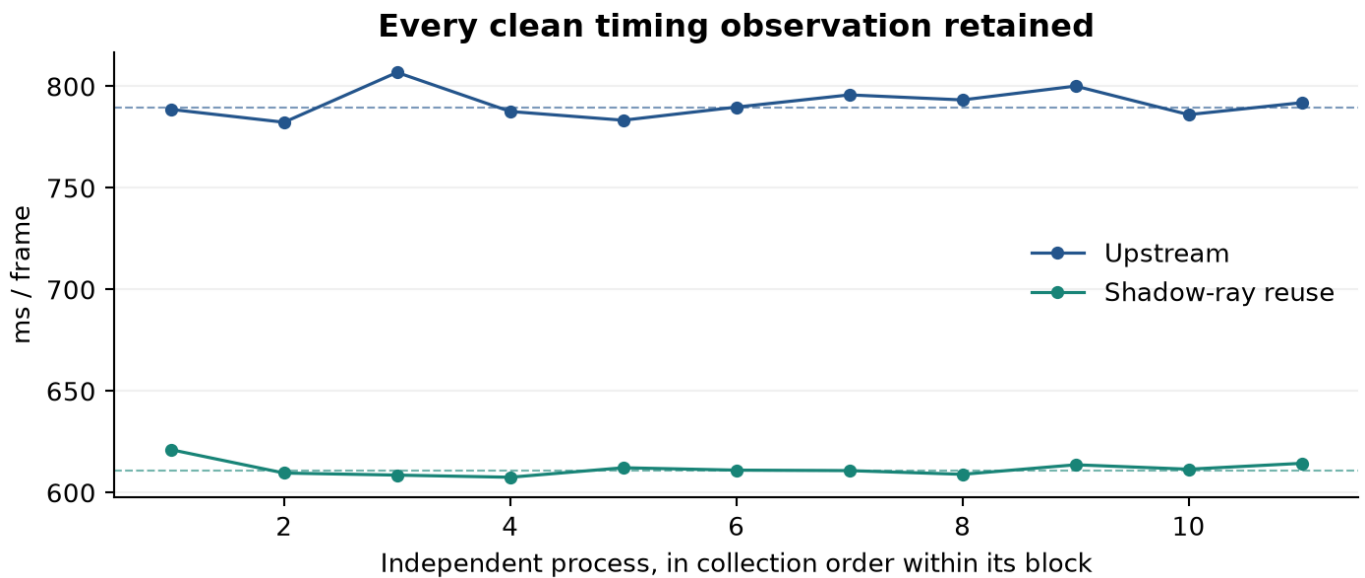
Candidate	Change / reason	Status
guards	Stock-class fast paths avoid polymorphic calls; preserve original fallback for other objects.	Included; alone +1.31% time locally.
sphere_scalar	Inline scalar dot/subtraction arithmetic; remove temporary objects; retain expression order.	Included; alone -14.64% locally.
camera	Reuse column/row camera components without reordering arithmetic.	Included; alone -1.07%.
nearest_hit	Direct selection instead of intermediate hit list/tuples; retain evaluations, ties and EPSILON.	Included; alone -2.14%.
checkerboard	Remove discarded Vector result while preserving reciprocal and scalar arithmetic/exceptions.	Included; alone +1.25%.
full	Combine those candidates with shadow-ray reuse.	Default; -38.55% locally.
slots / full_slots	Fixed attribute layout replaces stock geometry classes; different object-model contract.	Optional; full_slots -39.81%.

These local results use CPython 3.14.6 on arm64 macOS, 11 rotating repetitions and two 100 x 100 frames per sample. They are exploratory, not Ubuntu measurements; small individual regressions are not resolved reliably. Effects are not additive, and a faster combination does not prove every component helps. Keep explicit kernels for target-VM ablation.

Strict correctness means bit-identical benchmark pixels and preserved tested computation at successful boundaries. Tests include raw float bits for 1,000 randomized sphere cases and edge cases, camera components, prequantization colours, varied scenes, custom fallbacks, ties and restoration. All four production-sized frames match in the saved full-batch gate. Twenty-four raytrace tests passed; all 63 combined software tests passed.

Rejected or deferred: unconditional guard deletion breaks polymorphism; exact-class rejection without fallback narrows behavior; fixing upstream shadow/checkerboard semantics changes the task. Lower precision, approximate roots, a new integrator or answer caching violate the exact reference contract. Compiled native backends remain a possible separate experiment, outside the agreed pure-Python implementation.

Runtime clears the target; counters explain why



Eleven independent processes per arm, four frames per process. Dashed lines are medians. Every observation is retained; arms were collected in separate blocks.

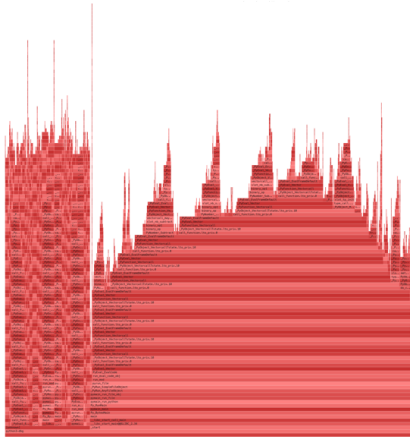
Metric, normalized per frame	Upstream	shadow_ray	Change
Median clean runtime	789.756 ms	611.023 ms	-22.63% / 1.2925x
Mean +/- sample SD	791.416 +/- 7.344 ms	611.757 +/- 3.745 ms	CV 0.93% / 0.61%
Instructions (perf TXT)	5.068 billion	3.892 billion	-23.20%
Cycles (perf TXT)	1.915 billion	1.487 billion	-22.38%
Branches / L1 data loads	653.0m / 1,639.1m	497.2m / 1,263.4m	-23.85% / -22.92%

Runtime, instructions, cycles and calls move together, as predicted by removal of repeated setup. A separate perf CSV batch also shows about 23% fewer instructions; do not merge it with TXT samples. L1 misses decrease 13.38%, yet their rate rises from 2.65% to 2.98% because loads fall faster. A worse rate does not mean more misses or a slower program.

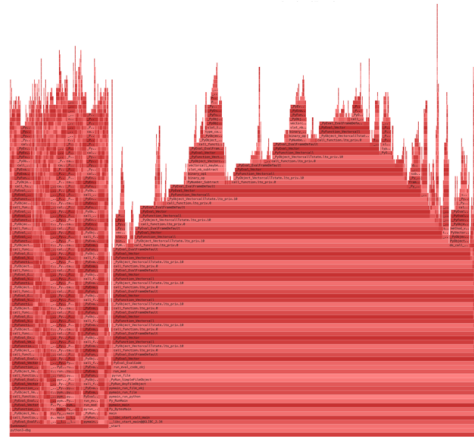
All optimized timing observations beat all baseline observations. The effect is much larger than within-block variation, although VM drift remains unmeasured. The official pyperformance baseline reports about 800 +/- 5 ms, consistent with the custom baseline. It is a cross-check, not an official optimized result.

Why the flame graphs still look similar

Upstream



shadow_ray



Structural overview of the original baseline and shadow-ray native flame graphs. Each panel is independently normalized. Use the adjacent original SVG files to zoom; the count table below is the quantitative comparison.

The same interpreter, render algorithm, recursion and geometry methods still run. Reusing shadow rays removes repeated executions; it does not remove the renderer from the call tree. Both SVGs fill the available width, so a faster run can have a similar outline. Debug wrapper ancestry can also deepen a stack independently of useful workload.

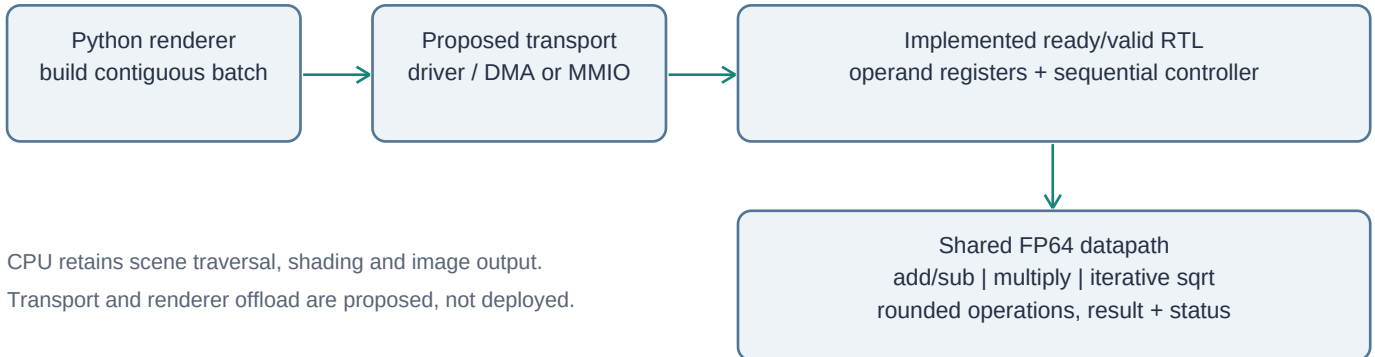
Equal-work native sample observation	Before -> after	Meaning
All sampled stacks	1,073 -> 823	23.30% fewer samples at the same cycle period.
Evaluator self	320 -> 227	Less evaluator work in absolute samples.
call_function self	47 -> 22	Consistent with fewer Python calls.
frame_dealloc self	33 -> 27	Absolute work falls; share rises 3.08% -> 3.28%.
Average stack depth	67.30 -> 69.25	Not evidence of more work or a regression.

The top-30 stacks cover only 297/1,073 and 244/823 samples; they cannot stand in for whole-profile categories. Unknown frames occur in roughly one-fifth of paths, and sparse symbol changes are uncertain. The normalized differential graph shows redistribution of share, not absolute saved time per frame. Filtering native stacks for Python symbols does not turn them into Python-level py-spy samples.

Remaining work includes sphere/plane queries, recursive shading, attribute access and boxed arithmetic. The next full-kernel VM run must measure how much of these costs remains after scalarization. Do not apply an Amdahl bound derived from the old debug profile to the new release kernel.

A small, explicit FP64 accelerator

The proposal offloads a complete ray-sphere intersection rather than individual floating-point instructions. This boundary can remove Python objects, calls and dispatch as well as execute arithmetic.



Implemented accelerator boundary is the ready/valid RTL. The host transport is an integration proposal.

`hardware/rtl/ray_sphere_accel.sv` contains the sequential controller, operand/result registers, status and ready/valid protocol. The datapath uses actual unmodified BSD-licensed Berkeley HardFloat Release 1 RTL: one shared add/sub unit, one multiplier and an iterative square-root unit. There are no real-number or software-math placeholders in the synthesizable datapath.

```

cp = center - origin
v = (cp.x*dx + cp.y*dy) + cp.z*dz
q = (cp.x*cp.x + cp.y*cp.y) + cp.z*cp.z
disc = r*r - (q - v*v)
result = no_root if disc < 0 else v - sqrt(disc)
  
```

Each operation rounds separately to binary64, nearest-even, including gradual underflow. There is no FMA, reassociation, FP32 or approximate reciprocal root. A tangent and a negative returned distance preserve upstream behavior; software applies its original hit-selection rule. Finite inputs/intermediates, nonnegative radius and caller-normalized direction define the supported domain. Invalid/nonfinite cases return an error for CPU fallback.

HardFloat source and license: `hardware/vendor/` provenance and license files. Primary specification: <https://www.jhauser.us/arithmetic/HardFloat-1/doc/HardFloat-Verilog.html>. Detailed ports, schedule and host format: `docs/HARDWARE_DESIGN.md` and `hardware/`.

Interfaces, software stack and honest estimates

Item	Specification / status
Request	640 operand bits (10 FP64 values) plus 32-bit tag; ready/valid; one outstanding query.
Response	64-bit distance, tag, root/error status and 5 FP exception flags; stable under backpressure.
Control	Reset discards in-flight work; explicit sequential FSM; completion returns to accepting state.
Software	Host packing/reference helper exists. Batched renderer scheduling, driver and transport/DMA are proposed, not deployed.
Frequency / schedule	50 MHz target assumption; 20-cycle miss / 80-cycle root initiation budgets. Not synthesis or simulation results.
Validation	Syntax/elaboration and host-format checks only. No HDL behavioral simulation, timing analysis or physical measurement.

Illustrative pessimistic case: 179,457 queries/frame, all taking a root; 50 MHz; 1 GB/s transport; batches of 1,024 with 10 us launch cost; 100 ns packing/query. The model gives 287.13 ms computation + 18.66 ms transfer + 1.76 ms launch + 17.95 ms packing = 325.50 ms offload cost. These are assumptions, not measured hardware performance.

Use $T_{\text{new}} = T_{\text{software}} \times (1 - f) + T_{\text{offload}}$, where f is the release-time fraction actually removed by a real batch implementation. Starting from the measured 611.02 ms shadow-ray software, break-even needs $f > 53.27\%$. With $f = 50\%$, the estimate is 631.01 ms: 3.27% slower. With $f = 70\%$, it is 508.81 ms: 16.73% lower time. Neither f is established by the debug profile, and the newer full software requires a new baseline.

A single shared datapath limits area and switching relative to many parallel lanes, but limits throughput; FP64 sqrt/rounding still costs substantial logic. Deeper pipelining or replication trades area, clocking and power for throughput. No LUT, gate, watt or achieved-clock numbers are claimed. The proposal needs batch amortization and must eventually be compared with an equivalent compiled CPU batch, since bypassing Python alone may explain most gains.

The assignment requires a complete logical HDL design and interface/architecture estimates (pp. 4-5), not physical implementation. Source completeness and syntax checks do not prove functional correctness. Hardware numerical/pixel acceptance and real transport integration remain future work.

What is established and what comes next

Shadow-ray reuse is an exact, measured improvement over the upstream workload and exceeds the declared 7% runtime goal. Matching call-count reductions, approximately 23% fewer instructions and the 22.63% runtime reduction explain the gain more convincingly than a broad label such as interpreter-bound. Similar flame shapes are compatible with fewer executions of the same code.

The full exact software now contains the remaining implemented candidates and passes the expanded correctness suite. Its Mac ablation is evidence for further testing, not a claim of target-VM improvement. Run the full workflow on Ubuntu and retain the selected session manifest before revising this report. Keep individual kernels available to identify components that regress there.

```
./run_all.sh
# Defaults: nbody flat_pow; raytrace full; both baseline arms.
./script_raytrace.sh --variant both --kernel full --loops 4
python3 -B variants/bm_raytrace_upstream_opt.py \
--mode verify --loops 4
python3 -B -m unittest discover -s tests -v
```

The scripts verify the actual batch, preserve authored reports, record source hashes and list missing optional tools. Session manifests name the exact baseline, optimized and comparison directories; they do not silently pair arbitrary latest folders. The balanced A/B timing tool is available for confirmation when a result is close or ordering effects are suspected.

Submission component	Location / status
Required six-section TXT report	report_raytrace.txt, exported from the same content as this PDF.
Scripts and README	script_raytrace.sh, run_all.sh, README.md.
Code / validation / experiments	variants/, tests/, docs/evidence/, docs/RAYTRACE_FULL_EXPERIMENT.md.
Profiles and raw results	Selected results/ directories; docs/reports/evidence_registry.json; original flame SVGs in assets/.
Hardware architecture and HDL	hardware/ and docs/HARDWARE_DESIGN.md; limitations stated above.
AI prompts / presentation	prompt.txt records actual prompts. Presentation (20-25 min plus questions), live demo and repository upload remain separate submission actions.

Course source: Project.pdf, pp. 3-5 and 7-8; requirement mapping in docs/PROJECT_REQUIREMENTS.md. No Git operations or repository publication were performed in this work.